WarpX SEE Sheath Project — Onboarding & User Guide (CHANL)

Author: Shelby Horth Date: August 31, 2025

Purpose. This document is a lasting, group-wide reference: part onboarding manual, part simulation workflow, and part physics guide for Secondary Electron Emission (SEE) sheaths in WarpX. It captures the steps, pitfalls, and fixes I encountered, with runnable templates and figures.

Contents

1 Preface (Purpose & Context) **3**

2 Executive Summary (TL;DR) **3**

3 Physics Setup & Objectives **5**
- 3.1 Physical Picture: Electron-Emitting Wall and Sheath . . . 6
- 3.2 Guidance from Hobbs & Wesson's SCL Regime . . . 7

4 Environment, Build & Runtime (Puma + Local) **8**
- 4.1 Quick Start on Puma (module-provided WarpX) . . . 10
- 4.2 Building WarpX From Source (with HDF5 + ADIOS2) . . . 10

5 Input Deck Anatomy (What Each Knob Means Physically) **13**

6 Diagnostics: What to Turn On & Why **17**
- 6.1 Full Diagnostics (openPMD) . . . 18
- 6.2 `diag_scrape` (Boundary Scraping) . . . 18
- 6.3 Potential from Field Integration . . . 18

7 Post-Processing & Plotting **18**
- 7.1 Python Environment . . . 19
- 7.2 Fields & Potential (Script Template) . . . 19
- 7.3 Phase Space (z–p_z) . . . 20
- 7.4 Scrape Counts vs. Time . . . 21
- 7.5 ParaView (remote) . . . 22

8 Troubleshooting & Pitfalls (What Broke & Fixes) **22**

9 Reproducible Run **23**
- 9.1 Minimal 1D SEE Sheath (fast turnaround) . . . 23
- 9.2 Scaling Up . . . 23
- 9.3 Long Run (10k iterations) . . . 23

10 Folder & File Keeping **23**

11 Documentation Figures **24**

12 Open Questions & Next Steps **29**

13 Appendix A — Example "Everything in One Place" Slurm **30**

14 Appendix B — Common Errors (with fixes) **30**

List of Figures

1 SEE sheath schematic: expected phi(z) and Ez(z) trends . . . 7

2 Schematic diagram of an SEE–modified DC sheath and virtual–cathode formation . 8

3 E_z profile ($\sim 0.03\,\mathrm{ns}$) . . . 20

4 SEE electron phase space at iteration 2890 21
5 S1. Electron phase space (early triangular plume) 24
6 S2. Electron charge density line-out (0.03 ns) 25
7 S3. E_z line-out (0.03 ns, extended domain) 26
8 S4. Electron phase space (later snapshot) 27
9 S5. Electron charge density with sharp spike 28
10 E_z at $\sim 0.15\,\mathrm{ns}$ 29

1 Preface (Purpose & Context)

This guide is meant to be a single place for:

- The **physics** behind SEE-driven direct current (DC) sheaths (with practical modeling notes).
- The **workflow** to compile, run, and post-process WarpX jobs on **Puma** (UArizona HPC) and locally.
- The **input-deck anatomy** (what each knob means physically) and working templates.
- The **diagnostics** you should turn on and how to read them (with plots and scripts).
- A curated list of **pitfalls** I hit (and how to avoid or fix them fast).

2 Executive Summary (TL;DR)

This project aims to build and analyze a very simple but revealing plasma configuration: a slab of plasma adjacent to a material wall that emits electrons. In the Particle-In-Cell (PIC) code WarpX, we represent this as a one–dimensional domain along the normal direction z (you can extend to two dimensions later for transverse effects). The purpose is to observe how *secondary electron emission* (SEE) from the wall modifies the sheath—the thin, space–charge layer that forms near a boundary—so that we can recognize when the system transitions into regimes predicted by classical theory. In particular, we compare our simulation behavior against the expectations of Hobbs & Wesson's space–charge–limited (SCL) emission theory: as emission strengthens, the sheath can develop a local potential maximum (a "virtual cathode") and the electric field near the wall can reverse sign. Part of the learning goal is practical: you should be able to compile WarpX, run the case on a cluster or laptop, and reliably extract the three most important diagnostics for this problem: the normal electric field $E_z(z)$, particle phase space near the wall, and the rate at which particles hit (or are "scraped" by) the boundaries.

A single control parameter organizes much of the expected physics in these SEE sheath problems: the dimensionless ratio

$$r = \frac{\Gamma_{\mathrm{SEE}}}{\Gamma_e},$$

where Γ_e is the flux of electrons *arriving* from the plasma to the wall and Γ_{SEE} is the flux of secondaries *emitted* by the wall back into the plasma. If r is small, emission is weak and the sheath resembles the usual monotonic potential drop from the plasma to the wall. As r approaches unity and exceeds it, the negative space charge of the emitted electrons can build up close to the surface. This space charge reshapes the sheath: E_z near the wall can flip sign over a short distance (a "field reversal"), and the electrostatic potential $\phi(z)$ can develop a local maximum just off the wall—this is the "virtual cathode." You do not need to memorize formulas to recognize this regime. In practice you will (i) look for a change of sign in E_z within a few Debye lengths of the wall, (ii) reconstruct $\phi(z)$ by integrating the measured E_z profile and check whether it is non–monotonic, and (iii) confirm that the boundary interaction rates and the near–wall particle distribution are consistent with stronger emission.

To run the first simulation, think in terms of a minimal, controlled experiment. Use a 1D z–domain with the emitting boundary at $z = 0$ and a simple absorbing (or reflecting) boundary at $z = L_z$ to represent the plasma side. In your WarpX input deck, this means defining the physical box in meters (`geometry.prob_lo` and `geometry.prob_hi`), choosing a grid resolution fine enough

to resolve a fraction of the Debye length, and selecting field and particle boundary conditions that match the picture above (conducting at the wall; absorbing on the far side is a good start). Populate the domain with plasma electrons or inject them from the plasma–side boundary; then enable an SEE source at the wall either as a prescribed flux (how many emitted electrons per unit area per unit time) or via a yield model (how many secondaries per incident primary). When you press "go," make sure you are saving the normal electric field E_z on the grid, capturing a representative sample of electron particle data to build z–u_z phase–space plots near the wall, and recording a boundary "scrape" metric that counts how many particles hit each boundary over time. Those three outputs—E_z, phase space, and scrape counts—are sufficient to tell you whether emission is turning on and how the sheath is responding.

Behind the scenes, WarpX writes diagnostics using the *openPMD* standard, which is simply a well–defined format and directory layout for field and particle data. For this project we strongly prefer the HDF5 backend because it works smoothly with common analysis tools (e.g., `openPMD-viewer`, `yt`, ParaView). In your inputs you'll see a line such as `warpx.openpmd_backend = h5`; that setting controls the file format, but it only works if WarpX was compiled with HDF5 support. Alongside the full field/particle dumps, it is helpful to enable a reduced diagnostic that focuses on boundary interactions—often called `diag_scrape`—which produces lightweight files (commonly JSON) that list how many particles were removed at each boundary at a given iteration. With these two classes of outputs (full openPMD data and reduced scrape data), your post–processing tasks become straightforward: plot $E_z(z)$ at a few times, integrate it once to estimate $\phi(z)$ up to a reference constant, scatter–plot z versus the normal particle velocity u_z to visualize the emitted population, and graph the scrape counts versus iteration to see when SEE becomes significant.

New users often encounter a handful of recurring snags, and it helps to know what they look like and why they occur. The first is a build–time issue: if WarpX was compiled without HDF5 support, the code still runs but the most convenient full diagnostics are unavailable, which in turn makes some analysis scripts fail or become awkward. The fix is to rebuild with HDF5 enabled (and to keep the input line selecting the HDF5 backend). The second is a surprise with the reduced boundary diagnostic: the scrape files sometimes "appear late" in the run. This is usually not a physics problem; it reflects the *cadence* you set (e.g., every 50 or 100 iterations), the precise trigger conditions for writing, and, in some versions, buffering that flushes only after several intervals. If you are testing early–time behavior, write the scrape diagnostic more frequently and confirm that it is bound to the emitting boundary you care about. A third issue is a hard crash (a segmentation fault) that shows up when you aggressively raise the number of macroparticles per cell or the injection flux/velocity to "make things look nicer." In PIC, more particles reduce noise but also increase memory and communication costs; if you overshoot the machine's memory or create severe load imbalance across MPI ranks, the job can crash. Walk these settings up gradually, increase the number of MPI ranks, and use smaller mesh tiles (`amr.max_grid_size`) to spread memory. Another common point of confusion is plotting: it is easy to accidentally label the horizontal axis as z when the file actually holds x, or to flip the sign convention for E_z so that a field reversal appears to be in the wrong place. Always pull coordinates from the metadata provided with the field and be explicit about your sign and reference choices when integrating E_z to get $\phi(z)$. Finally, if your job seems unstable or hangs when scaling to multiple nodes, check your MPI/OpenMP settings: start with one OpenMP thread per rank, use the site–recommended MPI transport (and environment variables), and only increase threading once the basic run is stable.

If you keep these ideas in mind—what r means physically, how to construct the minimal domain, which three diagnostics to capture, and how to avoid the usual build/runtime pitfalls—you will be able to reproduce the hallmark signatures of SEE–modified sheaths. From there, it is natural to extend the application: sweep the emission strength to watch the onset of field reversal, lengthen

the run to see whether the system approaches a steady or oscillatory state, and, when ready, add a second dimension to test how robust the conclusions are beyond 1D.

---

3 Physics Setup & Objectives

A Deep–dive (plain-language walkthrough)

This project models the thin layer of plasma that naturally forms next to a solid surface, called a *sheath*. Our specific twist is that the surface emits electrons: when energetic electrons from the plasma hit the wall, the impact knocks out additional, lower-energy electrons from the material. This process is *secondary electron emission* (SEE). In WarpX we start with the simplest possible geometry so that every feature you observe is easy to interpret. Imagine a straight line running normal to the wall; call that direction z. The wall sits at $z = 0$ and the plasma fills the region $z > 0$. A one-dimensional simulation (1D in z) is already enough to capture the essential sheath physics. Later, you may extend the domain to two dimensions (for example, x–z) to check that your conclusions survive when particles can move sideways, but the story begins in 1D.

The plasma close to the wall must reorganize itself to satisfy two constraints: Poisson's equation, which links the electric potential to any excess charge, and overall current balance, which ensures that charges are not piling up indefinitely at the boundary. When emission is weak, the electric potential typically decreases smoothly from the plasma toward the wall, repelling most hot electrons and attracting any ions. As the emission strengthens, however, the cloud of slow, newly emitted electrons can become dense enough to create a local region of negative space charge just off the surface. That additional charge bends the potential curve upward to form a *local maximum* (often called a *virtual cathode*) and flips the sign of the normal electric field E_z over a short distance. Recognizing the onset of this "field reversal" is one of the central goals of the study.

A single, intuitive ratio helps you anticipate which regime you are in. Denote by Γ_e the flux of electrons that *arrive* at the wall from the plasma (number per area per time), and by Γ_{SEE} the flux of electrons that the wall *emits* back into the plasma due to SEE. Define

$$r \;=\; \frac{\Gamma_{\mathrm{SEE}}}{\Gamma_e}.$$

When r is small, emission is a minor perturbation and the sheath remains monotonic. As r approaches 1 and exceeds it, the emitted electrons can dominate the local charge balance; a virtual cathode forms and E_z near the wall reverses sign. In practice you do not need to know Γ_e with laboratory precision. You will infer increasing r from simple simulation observables: a surge in particles "scraped" at the wall, a thickening band of near-wall electrons in phase space, and the appearance of a non-monotonic potential reconstructed from the measured field.

To keep the first runs simple and robust, we make a few modeling choices. We treat the fields electrostatically, meaning we solve for the electric potential and field without evolving electromagnetic waves. This is well matched to a slowly varying DC sheath. We begin with only one kinetic species—electrons—and hold ions as a neutralizing background; this avoids slow ion time scales while you learn the electron physics. At the boundary $z = 0$ we prescribe SEE either by specifying a *flux* (how many secondaries per area per time) or by using an effective *yield* model (how many secondaries per incident primary, often with a small emission temperature of order a few eV). All of these choices are easy to relax later: you can enable a magnetic field, give ions dynamics, swap in a more detailed yield curve, or move to 2D once the 1D behavior is well understood.

Before launching a simulation, check that the numerical mesh and time step can resolve the natural plasma scales that govern sheath structure. The most important spatial scale is the Debye length,

$$\lambda_D \;=\; \sqrt{\frac{\varepsilon_0 k_B T_e}{n_e e^2}},$$

which characterizes how quickly the plasma screens electric fields. Choose the grid spacing Δz to be a modest fraction of this, a common rule being $\Delta z \lesssim 0.3\,\lambda_D$. Make the domain long enough to contain both the sheath and a bit of "bulk" plasma; a thickness of at least $\sim 10\,\lambda_D$ is a safe starting point. The most important time scale is the electron plasma period, $2\pi/\omega_{pe}$ with $\omega_{pe} = \sqrt{n_e e^2/(\varepsilon_0 m_e)}$. Pick a time step small compared to this (a conservative guideline is $\Delta t \lesssim 0.2/\omega_{pe}$) and also respect the code's Courant–Friedrichs–Lewy (CFL) limit. If you do this, the sheath will form smoothly without numerical instabilities masquerading as physics.

When you post-process a run, focus on four visual cues that together tell a coherent story. First, inspect the normal electric field $E_z(z)$ close to the wall; as emission strengthens you should see a small positive lobe give way to a sign change over just a few grid cells. Second, integrate the measured E_z to reconstruct the electrostatic potential $\phi(z)$ up to an arbitrary constant:

$$\phi(z) \;=\; \phi(0) \;-\; \int_0^z E_z(\zeta)\,d\zeta.$$

A clear, local maximum of ϕ within a few Debye lengths of the wall is the hallmark of a virtual cathode. Third, look at the electron phase space near the wall (a scatter or density plot of z versus the normal velocity or momentum u_z). A visible population of slow, outward-moving secondaries hugging the surface—and gradually spreading due to space-charge effects—signals increasing r. Finally, plot the boundary "scrape" counts versus iteration. A sustained rise in the rate at which particles are removed at the wall confirms that the boundary interaction is changing in step with the field and phase-space signatures.

Classical analyses by Hobbs and Wesson provide a useful mental template for what to expect as you vary the emission strength. In their space-charge-limited (SCL) picture, the emitted electron cloud is sufficiently dense to alter the sheath self-consistently: the potential is no longer a simple monotonic drop, and the electric field must reverse direction across a short region to accommodate the local maximum. In our simulations we use that theory qualitatively rather than literally. Geometry, boundary conditions, and injection details in a PIC model differ from the idealized assumptions in a textbook derivation. What matters is the *trend*: as you dial up emission, the potential acquires a peak near the wall and the sign of E_z flips across the near-wall band. Seeing those trends together is strong evidence that you have entered the SCL-like regime.

3.1 Physical Picture: Electron-Emitting Wall and Sheath

It helps to have a concrete picture. Picture a flat conducting wall at $z = 0$ and a quiet plasma to the right. Fast plasma electrons stream toward the wall and many are absorbed. Each impact can liberate one or more secondaries from the material, usually much colder than the incident primaries and launched roughly normal to the surface. Because these secondaries are slow, they do not travel far before the electric field acts on them. If enough are produced, they form a cushion of negative charge that depresses the potential locally. That depression competes with the usual sheath drop and can overshoot into a local maximum of $\phi(z)$ a small distance from the wall—the virtual cathode. Just as a ball rolling over a hill reverses its direction, the electric field must reverse sign across this crest to satisfy Poisson's equation and the boundary conditions. In the code, you

emulate this by enabling a surface source at $z = 0$ with a chosen flux, small emission temperature (a few eV is common), and initial drift slightly away from the wall.

Two practical knobs control how strongly this near-wall cloud develops. One is the emission strength itself (either the flux you prescribe or an effective yield that scales with the incoming current). The other is the energy (or temperature) you assign to the secondaries. Warmer secondaries spread out faster, diluting the space charge; very cold secondaries pile up near the surface and accentuate the virtual cathode. Start conservatively, observe the qualitative signatures, and then explore parameter scans to map where the field reversal first appears.

3.2 Guidance from Hobbs & Wesson's SCL Regime

Hobbs & Wesson's analysis is a compass for what your diagnostics should reveal as emission increases. In the SCL regime, the sheath organizes so that the total current at the wall balances: conduction current carried by particles plus any displacement current in the time-varying field must sum to zero in steady conditions. The emitted electron cloud contributes a space-charge term that forces the electric field to pass through zero and change sign close to the wall. Translating that into checks you can perform: when you plot $E_z(z)$, you should find a zero crossing within a few λ_D of $z = 0$ once r approaches unity; when you integrate E_z to obtain $\phi(z)$, you should see a local maximum at roughly the same location. The phase-space portrait should show a dense, cool population leaving the wall and, depending on parameters, some secondaries turning around near the potential crest. Meanwhile, the boundary diagnostics should show an uptick in scrape rates consistent with the stronger interaction at the wall. If those pieces move together as you increase the emission setting, you are witnessing the SCL-like transition the theory anticipates.

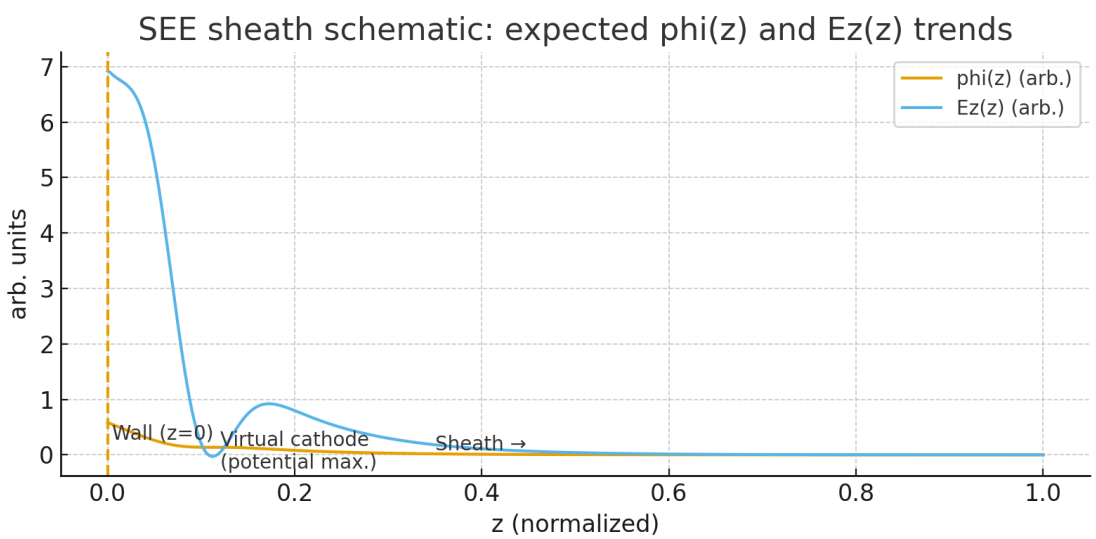

Figure 1: Conceptual geometry with wall at $z = 0$, near-wall potential maximum (virtual cathode), and field-reversal region. The plasma occupies $z > 0$. Emitted secondaries (light arrows) form a negative space-charge cloud within a few Debye lengths, bending the potential upward and forcing E_z to change sign. This is the qualitative behavior we look for as the emission ratio $r = \Gamma_{\mathrm{SEE}}/\Gamma_e$ approaches and exceeds unity.

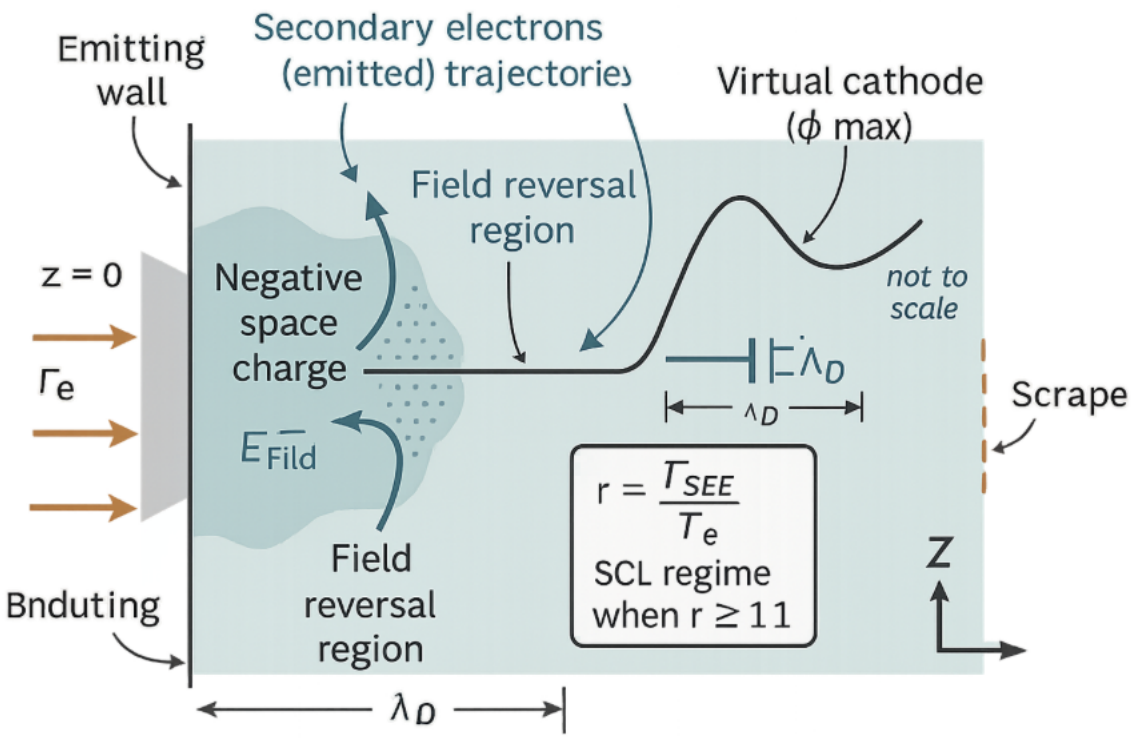

Figure 2: A conducting wall at z=0 emits secondaries (light arrows) when struck by incoming plasma electrons (dark arrows). Within a few Debye lengths λ_D, the emitted electrons form a negative space–charge cloud that reshapes the sheath: the normal field E_z reverses sign over a short region and the electrostatic potential $\phi(z)$ develops a local maximum (the "virtual cathode") at $z \approx z_{\mathrm{vc}}$. The far side of the domain represents the quasi-neutral plasma (absorbing or reflecting boundary at z=L_z). Annotations indicate the key control $r = \Gamma_{\mathrm{SEE}}/\Gamma_e$; as $r \to 1$ the scrape rate at the wall rises, a near-wall band appears in z–p_z phase space, and $E_z(z)$ shows the characteristic sign change. These are the primary diagnostics we track in WarpX: field line-out $E_z(z)$, reconstructed $\phi(z)$ from $\int E_z\, dz$, phase space near the wall, and boundary scrape counts.

---

4 Environment, Build & Runtime (Puma + Local)

A Step-by-step

If you are new to high-performance computing and WarpX, the safest route is to begin with whatever your cluster already provides and only build from source once you can run a small test. On **Puma** (UArizona HPC), there is typically a site module that exposes a ready-to-use WarpX executable; loading the module also configures paths and compilers so the code finds MPI and I/O libraries. Open a terminal on a login node, start a fresh shell, and load the module. You can confirm that the executable is visible by asking the shell where it lives; if you see a full path, the module is active

and you are ready to submit a small job.

```
module load warpx/25.03 # site-provided WarpX on Puma, example version
which warpx # should print a full path to the binary
```

For post-processing (plotting fields, reading particle data, etc.), it is convenient to add a Python environment that already includes the readers and plotting tools. Some clusters package this as a separate module; if Puma provides `hpcteam/pywarpx`, load it as well and test that the key imports succeed.

```
module load hpcteam/pywarpx/25.03
python3 -c "import yt, openpmd_api; print('ok')"
```

Most WarpX runs on Puma will launch under MPI. To avoid transport or fabric surprises the first time you scale out, set conservative OpenMPI environment variables and keep OpenMP threading at one thread per rank. This keeps the parallelism simple until you are confident the case is stable.

```
export OMPI_MCA_pml=ob1
export OMPI_MCA_btl=self,tcp
export OMP_NUM_THREADS=1
```

With the environment in place, try a *smoke test*. The goal is not to produce science yet—only to check that the code starts, advances a few dozen steps, and writes diagnostics. Submit a tiny Slurm job (one node, one to four ranks) and cap the run at, say, 50 iterations. If you see diagnostic directories appear (for example, `diags/diag000100` for openPMD output and `diags/diag_scrape/` for reduced JSON), your build and run path are healthy.

```
# inside a Slurm script or an interactive allocation
srun -n 1 $(which warpx) -i inputs_sheath_1d max_step=50
```

Once the smoke test works, you can scale gradually. On Slurm, increasing `--ntasks-per-node` adds MPI ranks; reducing `amr.max_grid_size` in the input deck tells WarpX to subdivide the mesh into smaller boxes so memory and work distribute across ranks. Expect less-than-linear speedup at first; the goal is simply to shorten wall-clock time without stressing memory.

Building with HDF5 (why & how)—in plain language

WarpX writes its fields and particles using the *openPMD* standard. Several tools you will use (`openPMD-viewer`, `yt`, and ParaView) are most dependable when the files are in the **HDF5** format. If your site module was compiled without HDF5, you can still run, but you will find yourself fighting I/O limitations. The remedy is to compile WarpX with HDF5 enabled and then select the HDF5 backend in your input file. After a successful build, the command `warpx --version` will usually list which backends are compiled in; seeing "openPMD: HDF5" (or similar) is the confirmation you need. In the input deck, set

```
warpx.openpmd_backend = h5 # or "hdf5", depending on the WarpX version
```

so that new runs write HDF5 files. When HDF5 is enabled, full diagnostics integrate cleanly with your Python plotting scripts and ParaView, which removes a major bottleneck from the workflow.

Runtime sanity checks

When you run on a small problem size, watch for three quick indicators of health. First, diagnostic folders should appear at the cadence you specified; if your reduced boundary diagnostic (`diag_scrape`) seems to "start late," it is usually because the write interval is large or a buffer is flushing every few intervals—shorten the interval during early-time studies. Second, increasing the number of MPI ranks should reduce time per iteration, even if not perfectly linearly. Third, scan the standard output for warnings like "NaN detected"; if you see them in the first few hundred steps, back off the emission strength, reduce the time step, or simplify the boundary conditions until the numerical setup is stable.

4.1 Quick Start on Puma (module-provided WarpX)

If a module is available, it is the quickest path from zero to plots. Load the module, verify the binary path, and, if needed, add the plotting environment. The three commands below are often everything you need before you submit a first job.

```
module load warpx/25.03 # or a newer site-provided version
module list # inspect what is loaded
which warpx # confirm the binary is on PATH
```

If you plan to analyze results on the cluster, add the site Python environment and verify imports:

```
module load contrib
module load hpcteam/pywarpx/25.03
python3 -c "import yt, openpmd_api; print('ok')"
```

Keep MPI and threading simple until you have a stable configuration:

```
export OMPI_MCA_pml=ob1
export OMPI_MCA_btl=self,tcp
export OMP_NUM_THREADS=1
```

If your site prefers UCX/InfiniBand transports, staff may recommend different OpenMPI variables; adopt those once your basic run behaves.

4.2 Building WarpX From Source (with HDF5 + ADIOS2)

If the module does not include HDF5, or if you want a self-contained toolchain you can reproduce, compiling from source inside a **Conda** environment is a reliable path. Conceptually you will (i) create an isolated environment with compilers and libraries, (ii) point CMake at those compilers, (iii) enable MPI and openPMD/HDF5 (and optionally ADIOS2), and (iv) build and install. Below is a minimal outline for reference; the exact library names and versions may vary with time.

Typical dependencies. You will need HDF5 (parallel), ADIOS2 (optional but useful for ParaView and some workflows), FFTW (optional), and the openPMD API. On Puma, you can either load module versions of these before configuring CMake, or install them into your Conda environment.

Generic CMake outline.

```
# Configure in a fresh build directory
cmake -S /path/to/warpx -B build \
  -DWarpX_PYTHON=ON \
```

```
  -DWarpX_MPI=ON \
  -DWarpX_OPENPMD=ON \
  -DopenPMD_USE_HDF5=ON \
  -DWarpX_FFT=ON \ # optional
  -DWarpX_QED=OFF \ # not needed here
  -DCMAKE_BUILD_TYPE=Release

cmake --build build -j 8
```

If you prefer to lean on site modules for the heavy libraries:

```
module load cmake gcc openmpi hdf5-parallel adios2
# Then run the cmake command shown above
```

If you want a fully self-contained Conda build:

```
conda create -n warpx-env python=3.11 -y
conda activate warpx-env
conda install -c conda-forge cmake make ninja mpi4py openmpi hdf5 adios2 openpmd-api -y
# Then run the cmake command shown above; CMake will discover the conda toolchain
```

At runtime, select the HDF5 backend in your input file so the diagnostics are written in the preferred format:

```
warpx.openpmd_backend = h5
```

Why this fixes a bottleneck. Earlier, the absence of HDF5 support forced us to rely on reduced diagnostics and ad-hoc readers; enabling HDF5 unlocks full openPMD dumps that tools like `yt` and `openPMD-viewer` read natively, making field integration for $\phi(z)$ and particle phase-space plots straightforward.

My exact Conda-based build (as used on Puma)

The following transcript captures the precise steps that succeeded when the generic instructions needed a few adjustments. If you are new, you can follow this verbatim; it produces both a WarpX executable for Slurm and Python bindings usable from `picmi`/`pywarpx` scripts.

1) Install Miniconda and create the toolchain environment.

```
# Create environment with compilers and core libs
conda create -n warpx-env -c conda-forge \
  python=3.11 gcc_linux-64 gxx_linux-64 cmake openmpi hdf5 fftw adios2 \
  openblas lapack git -y
conda activate warpx-env

# Add plotting and I/O Python packages
conda install -c conda-forge periodictable picmistandard h5py yt matplotlib notebook -y
```

2) Clone WarpX and make a build directory.

```
git clone https://github.com/ECP-WarpX/WarpX.git ~/warpx
cd ~/warpx
mkdir build && cd build
```

3) Point CMake to the Conda compilers and configure.

```
export CC=$(which x86_64-conda-linux-gnu-gcc)
export CXX=$(which x86_64-conda-linux-gnu-g++)
cmake -DCMAKE_C_COMPILER=$CC -DCMAKE_CXX_COMPILER=$CXX \
  -DAMReX_MPI_THREAD_MULTIPLE=off -DWarpX_MPI=ON -DWarpX_HDF5=ON \
  -DWarpX_ADIOS=ON -DopenPMD_USE_PYTHON=ON \
  -DPython_EXECUTABLE=$(which python3) \
  -DCMAKE_INSTALL_PREFIX=$CONDA_PREFIX \
  -DWarpX_DIMS="1;2;RZ;3" -DUSE_PYTHON_MAIN=TRUE \
  -DPYINSTALLOPTIONS=$CONDA_PREFIX/lib/python3.11/site-packages/ \
  -DWarpX_PYTHON=ON ..
```

4) Build and install.

```
make -j$(nproc)
make install
```

This installation produces a WarpX executable you can point to from a Slurm script *and* installs the Python pieces into your active Conda environment so you can run PICMI/pyWarpX workflows.

Running on Slurm with the Conda build

If you are launching a Python PICMI input (a common pattern when prototyping), activate your environment inside the job and make sure the Python process can see the installed WarpX bindings. If you installed into the environment as shown above, the `PYTHONPATH` step may be unnecessary; if you did a local build without `make install`, export `PYTHONPATH` to point at the build's `site-packages` directory.

```
#!/bin/bash
#SBATCH --job-name=WarpX_conda
#SBATCH --account=hanquist
#SBATCH --partition=standard
#SBATCH --mail-type=FAIL,END
#SBATCH -o %x.%j.out
#SBATCH -e %x.%j.err
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=2
#SBATCH --mem-per-cpu=5gb
#SBATCH --time=1:00:00

cd $SLURM_SUBMIT_DIR
date

# Activate Conda
source ~/miniconda3/etc/profile.d/conda.sh
conda activate warpx-env

module load contrib
module load hpcteam/pywarpx/25.03
/contrib/hpcteam/pywarpx/env/bin/warpx.1d.MPI.OMP.DP.PDP.OPMD.EB.QED
    inputs_test_1d_scrape
date
```

On clusters where Slurm integrates tightly with MPI, replace `mpirun` with `srun`; both patterns work on Puma, but your center's documentation may prefer one. If you compiled a native WarpX executable (e.g., `warpx.1d.MPI.OMP.DP.OPMD`), you can also launch it directly instead of a Python driver:

```
srun -n $SLURM_NTASKS /path/to/warpx -i inputs_sheath_1d max_step=5000
```

Verifying the build. Two quick checks save hours later: run `warpx --version` and confirm that openPMD reports HDF5 support; then run a 50-step smoke test and confirm that a directory like `diags/diag000100` appears and can be opened by `openPMD-viewer` or `yt`. If either check fails, revisit the CMake flags enabling openPMD/HDF5 and ensure the environment that compiled WarpX is the same one you activate at runtime.

With this workflow you have (i) a cluster module path for fast starts, (ii) a reproducible Conda build that enables HDF5 and the Python ecosystem, and (iii) Slurm launch templates for both compiled inputs and PICMI scripts. From here, you can scale job size, tune MPI/OpenMP settings, and focus on the physics diagnostics (fields, phase space, and scrape counts) rather than the mechanics of getting the code to run.

5 Input Deck Anatomy (What Each Knob Means Physically)

Using the Deck

What follows is a plain-language tour of your input file, explaining what each group of parameters does physically and how the choices shape the SEE-sheath experiment.

Constants and species mass. You define a custom ion mass, `my_constants.m_no = 30.*m_u - 1.*m_e`, then set `m_no` and use it for the ion species. This is a practical way to model a 30 amu molecular ion (e.g. NO^+) while subtracting one electron mass for ionization. Using a realistic mass matters because it sets the ion plasma frequency and sound speed, which in turn influences how quickly the sheath can respond to boundary conditions on multi-nanosecond runs.

Domain and resolution. The mesh is one-dimensional (`geometry.dims = 1`) with `amr.n_cell = 512` cells spanning `z \in [0, 0.01636]~\mathrm{m}`, i.e. a 16.36 mm slab. This gives a uniform spacing $\Delta z \approx 3.19 \times 10^{-5}$ m (31.9 μm). In sheath studies, the spatial yardstick is the Debye length λ_D. The rule of thumb is $\Delta z \lesssim 0.3\,\lambda_D$ and a domain thickness of at least $\sim 10\,\lambda_D$. If your T_e and n_e imply a smaller λ_D than ~ 0.1 mm, either increase `amr.n_cell` or reduce `prob_hi` so that the sheath and pre-sheath are properly resolved.

Time step and numerics. You fix the time step with `warpx.const_dt = 0.05e-12`, i.e. $\Delta t = 5 \times 10^{-14}$ s (50 fs). That is comfortably small for most electron plasma frequencies you will meet in weakly ionized gas and easily satisfies the CFL condition for a 1D Yee advance (`algo.maxwell_solver = yee`). The comment "Do not evolve the E and B fields" is misleading when the Yee solver is selected—the fields *are* advanced; what you are doing in practice is a quasi-electrostatic experiment with a small Δt so that magnetic dynamics remain negligible. You set `warpx.serialize_initial_conditions = 1` to make multi-rank runs bitwise reproducible (initial particle placement is serialized), a good choice for debugging. The particle shape is declared twice (`algo.particle_shape = 1` early, then later

= 3); in ParmParse the *last* assignment wins, so you are running with a third-order B-spline (TSC). Higher shape order reduces grid-heating noise and is a sensible pick for sheath physics. You also disable digital filtering (`warpx.use_filter = 0`); keep an eye on noise and turn this on later if you see high-k ringing.

Boundary conditions and bias. Both field boundaries are perfect electric conductors (`boundary.field_lo = pec` and `..._hi = pec`). In addition, you set electrostatic Dirichlet values `electrostatic_potential_lo_z = -5.0` V at $z = 0$ and `..._hi_z = 0.0` V at $z = L_z$. Physically, this biases the left wall 5 volts negative relative to the plasma-side boundary. If you integrate a uniform field from right (0 V) to left (-5 V) over 16.36 mm, the mean field magnitude would be ~ 300 V/m pointing toward the wall. In a real sheath, the field is not uniform; it steepens within a few λ_D of the surface.
Implementation note: those explicit electrostatic potential keys are only active in the electrostatic solver; with the electromagnetic Yee solver they may be ignored by some versions of WarpX. If you rely on a fixed bias in EM mode, confirm in a smoke test (plot E_z near both boundaries) or switch to the electrostatic solver in inputs meant for DC sheaths.

Species, sources, and geometry of the experiment. You declare three kinetic species: `electron`, `ions`, and `see`. The geometry is a wall at $z = 0$ that emits secondaries (`see`), and a "plasma source" at the right boundary injecting counter-streaming electrons and ions. All three use the `NFluxPerCell` injection style, which is a surface *flux* boundary condition: at each time step and for each boundary cell on the chosen surface, the code creates a specified number of macroparticles consistent with the given flux and the local cell area.

- **Plasma electrons** are injected at $z = 0.01636$ m (`electron.surface_flux_pos` equals `prob_hi`) with their normal pointing in $-z$ (`electron.flux_direction = -1`). You set a constant flux profile (`parse_flux_function` with value "1."), and a Gaussian *flux* momentum distribution (`gaussianflux`) characterized by a mean `uz_m` and thermal spreads `ux_th`, `uy_th`, `uz_th`. The comment says the mean should be negative (flow right to left), but the value in the file is `electron.uz_m = 0.0022` (positive). For consistency, either make the number negative or keep it positive and change `flux_direction` to $+1$; otherwise you risk injecting electrons with a mean velocity that opposes the declared surface normal. The thermal width `uz_th = 0.04` produces a relatively broad velocity spread; transverse temperatures are set to zero here, acceptable in 1D but consider a small nonzero value to avoid numerical filamentation.
- **Plasma ions** are injected at the same right boundary with `flux_direction = -1` (toward $-z$) using a constant flux. Their Gaussian spreads are tiny (1.7×10^{-4}), and the mean `uz_m = 0.0022` matches the electrons so the plasma source is roughly current-neutral far from the sheath. Using a realistic mass m_{NO^+} slows ion response, which is desirable if you focus on sub-microsecond electron sheath dynamics.
- **Secondary electrons** (`see`) are emitted from the *left* wall (`see.surface_flux_pos = 0.0`) with `flux_direction = +1`, i.e. into $+z$. Their Gaussian flux has small transverse widths (0.01), a modest normal width (0.02), and a mean `uz_m = 0.07` away from the wall—this provides an initial outward push typical of cold secondaries with a few-eV effective temperature. In scans where you vary the SEE "strength," this mean and the flux profile are the principal knobs that move the ratio $r = \Gamma_{\mathrm{SEE}}/\Gamma_e$.

All three species use comparatively high macroparticle counts per boundary cell (`num_particles_per_cell = 500` for `electron` and `ions`; 100 for `see`). High ppc reduces statistical noise but raises memory and cost sharply. In 1D this is often fine, but in 2D you may need to

step these down or raise MPI ranks. A safe practice is to ramp ppc (e.g. 50 → 100 → 200) while keeping an eye on memory and time per step.

Diagnostics. You request a *Full* openPMD diagnostic named `diag1` every 100 iterations and ask it to write fields $E_x, E_y, E_z, B_x, B_y, B_z, \rho$. That is exactly what you need to reconstruct $\phi(z)$ from E_z and monitor charge build-up. The keys `diag1.format = openpmd` and `diag1.diag_type = Full` are the important ones; the extra line
`diag1.warpx_format = openpmd` is benign but redundant in newer releases. For boundary flux accounting, consider adding a reduced `diag_scrape` in this deck as you used elsewhere; it is much lighter than Full dumps and provides a quick look at Γ_e and Γ_{SEE} trends.

Putting it all together. Physically, this deck sets up a biased conducting wall at $z = 0$ that emits cold secondaries into the plasma, opposed by a right-hand plasma source that continuously injects electrons and ions toward the wall. As the run evolves, the near-wall potential adjusts to satisfy Poisson's equation and current continuity. By scanning the SEE flux or mean `uz_m` you can increase r and look for (i) a zero crossing of E_z within a few λ_D of the wall and (ii) a local maximum in the reconstructed $\phi(z)$: the virtual-cathode signature.

Your input deck

The listing below reproduces your file with line breaks added where two keys shared a line; *no values were changed*. You can paste this directly into a file and run.

```
# CONSTANTS
my_constants.m_no = 30.*m_u - 1.*m_e # subtract electron mass for ionization
m_no = my_constants.m_no
ions.mass = m_no

#################################
####### GENERAL PARAMETERS ######
#################################
max_step = 600
amr.n_cell = 512
amr.max_level = 0
amr.blocking_factor = 4
geometry.dims = 1
geometry.prob_lo = 0.
geometry.prob_hi = 0.01636

#################################
###### Boundary Condition #######
#################################
boundary.field_lo = pec
boundary.field_hi = pec
electrostatic_potential_lo_z = -5.0 # V (at z = 0)
electrostatic_potential_hi_z = 0.0

#################################
############ NUMERICS ###########
#################################
warpx.serialize_initial_conditions = 1
warpx.verbose = 1
```

```
warpx.const_dt = 0.05e-12
warpx.use_filter = 0
algo.maxwell_solver = yee
# Order of particle shape factors
algo.particle_shape = 1
################################
############ PLASMA #############
################################
particles.species_names = ions electron see
# (last value wins) use higher-order shape
algo.particle_shape = 3

electron.charge = -q_e
electron.mass = m_e
electron.injection_style = NFluxPerCell
electron.num_particles_per_cell = 500
electron.surface_flux_pos = 0.01636
electron.flux_normal_axis = z
electron.flux_direction = -1
electron.flux_profile = parse_flux_function
electron.flux_function(x,y,z,t) = "1."
electron.momentum_distribution_type = gaussianflux
electron.uz_m = 0.0022 # Negative sign is expected for left-going; verify
electron.ux_m = 0.0
electron.uy_m = 0.0
electron.ux_th = 0.0
electron.uy_th = 0.0
electron.uz_th = 0.04

ions.charge = +q_e
ions.mass = m_no
ions.injection_style = NFluxPerCell
ions.num_particles_per_cell = 500
ions.surface_flux_pos = 0.01636
ions.flux_normal_axis = z
ions.flux_direction = -1
ions.flux_profile = constant
ions.flux = 1.
ions.momentum_distribution_type = gaussianflux
ions.ux_th = 0.00017
ions.uy_th = 0.00017
ions.uz_th = 0.00017
ions.ux_m = 0.0
ions.uy_m = 0.0
ions.uz_m = 0.0022

see.charge = -q_e
see.mass = m_e
see.injection_style = NFluxPerCell
see.num_particles_per_cell = 100
see.surface_flux_pos = 0.0
see.flux_normal_axis = z
see.flux_direction = 1
see.flux_profile = parse_flux_function
```

```
see.flux_function(x,y,z,t) = "1."
see.momentum_distribution_type = gaussianflux
see.ux_th = 0.01
see.uy_th = 0.01
see.uz_th = 0.02
see.ux_m = 0.0
see.uy_m = 0.0
see.uz_m = 0.07

################################
############ COLLISION #########
################################
# collisions.collision_names = collision1 collision2 collision3
# collision1.species = ions ions
# collision1.CoulombLog = 10.0
# collision2.species = ions electron
# collision2.CoulombLog = 10.0
# collision3.species = electron see
# collision3.CoulombLog = 10.0

################################
############ DIAGNOSTICS ########
################################
diagnostics.diags_names = diag1
diag1.fields_to_plot = Ex Ey Ez Bx By Bz rho
diag1.format = openpmd
diag1.intervals = 100
diag1.diag_type = Full
diag1.warpx_format = openpmd
```

Checklist before long runs. (i) Verify that the electron source directions (`flux_direction` and `uz_m` sign) are consistent with right-to-left inflow; (ii) confirm whether the electrostatic potential keys are honored under your solver choice (plot E_z near both boundaries); (iii) if memory grows rapidly, reduce `num_particles_per_cell` or write diagnostics less frequently; (iv) consider adding a reduced `diag_scrape` to directly track Γ_e and Γ_{SEE} at each boundary.

6 Diagnostics: What to Turn On & Why

A Full (openPMD) fields/particles

- Fields: E_z, ρ, j_z. Particles: at least electrons' z, u_z, w.
- If storage is tight, keep particles off for survey runs; re-run with particles on after you find interesting regimes.

`diag_scrape` JSON anatomy

Typical entries include iteration index, wall ID, and particle counts/charge removed. If nothing appears early, either (i) cadence too coarse, (ii) wrong boundary, or (iii) the scrape buffer flushes only after N steps. Start with `diag_scrape.intervals = 25 to 50` to see early dynamics.

Reconstructing $\phi(z)$ from E_z

Discrete integral (uniform grid): $\phi_k = \phi_0 - \sum_{i=1..k} E_{z,i}\,\Delta z$.

- Choose ϕ_0 (e.g., 0 at the wall).
- Confirm axis direction (meta.z) to avoid sign flips.
- Optional noise-robustness: smooth E_z slightly (e.g., Savitzky–Golay) before integration.

What "onset of SEE" looks like

- Scrape: sustained rise above baseline.
- Fields: small near-wall positive E_z lobe, then sign reversal.
- Phase space: denser near-wall band with outward drift; broadening over time.

6.1 Full Diagnostics (openPMD)

- Turn on **fields**: E_z (primary), ρ, and currents (j_z) to reconstruct potential and monitor charge build-up.
- Turn on **particles** (if budget allows) to analyze energy and **phase space** (z–p_z or z–v_z).
- Prefer **HDF5 backend** for tooling compatibility; ADIOS2 works but some scripts/tools expect HDF5 readers.

6.2 `diag_scrape` (Boundary Scraping)

- Records count/flux of particles that hit the boundary—our proxy for Γ_e and Γ_{SEE} trends.
- **Observed "late start":** In earlier runs, scrape files appeared after **∼iteration 2200**. This was due to (a) cadence (`intervals`) and (b) triggers/thresholds in the reduced diag config. If you expect data earlier, lower the interval and confirm the diag is bound to the correct **boundary (zlo)**.

6.3 Potential from Field Integration

To estimate $\phi(z)$ from E_z:

1. Extract E_z over z at a given iteration.
2. Numerically integrate: $\phi(z_k) = \phi(0) - \sum_{i=1}^{k} E_{z,i}\,\Delta z$.
3. Choose a reference (e.g., set $\phi(0) = 0$). Watch your **sign convention** and axis direction! (This bit me once—plotting inversion.)

---

7 Post-Processing & Plotting

Batch plotting over all iterations

Loop template (bash):

```
for it in {100..5000..100}; do
  python3 scripts/plot_fields_phi.py $it || true
  python3 scripts/plot_phase_space.py $it || true
done
```

Modify the scripts to accept an optional iteration argument and the diag path from `sys.argv`.

Running plots in Slurm epilogue

Place the three plotting commands after the main `srun` in the provided Slurm template; keep them tolerant to missing iterations to avoid job failure.

ParaView client–server sketch

`pvserver --server-port=11111` on a compute node (interactive job), then connect from your workstation GUI: File → Connect → Add Server. Load openPMD/HDF5, create slices along z.

7.1 Python Environment

For local or Puma: create `warpx-post` env with **openPMD-viewer** and **yt**.

```
conda create -n warpx-post python=3.11 -y
conda activate warpx-post
pip install openpmd-viewer yt matplotlib numpy xarray h5py
```

If site modules exist:

```
module load hpcteam/pywarpx/25.03
python3 -c "import yt; import openpmd_api; print('ok')"
```

7.2 Fields & Potential (Script Template)

```
# save as plot_fields_phi.py
import numpy as np, h5py, matplotlib.pyplot as plt
from openpmd_viewer import OpenPMDTimeSeries

TS = OpenPMDTimeSeries('diags/diag000100') # adjust path
it = 3000 # iteration to plot
Ez, meta = TS.get_field(field='Ez', iteration=it)
# Assuming 1D in z
z = meta.z
# Integrate to get phi(z): phi(0)=0 reference
phi = -np.cumsum(Ez) * (z[1]-z[0])

plt.figure(); plt.plot(z, Ez); plt.xlabel('z [m]'); plt.ylabel('Ez [V/m]'); plt.title(f'
    Ez at it={it}'); plt.grid(True)
plt.figure(); plt.plot(z, phi); plt.xlabel('z [m]'); plt.ylabel('phi [V]'); plt.title(f'
    phi from Ez at it={it}'); plt.grid(True)
plt.show()
```

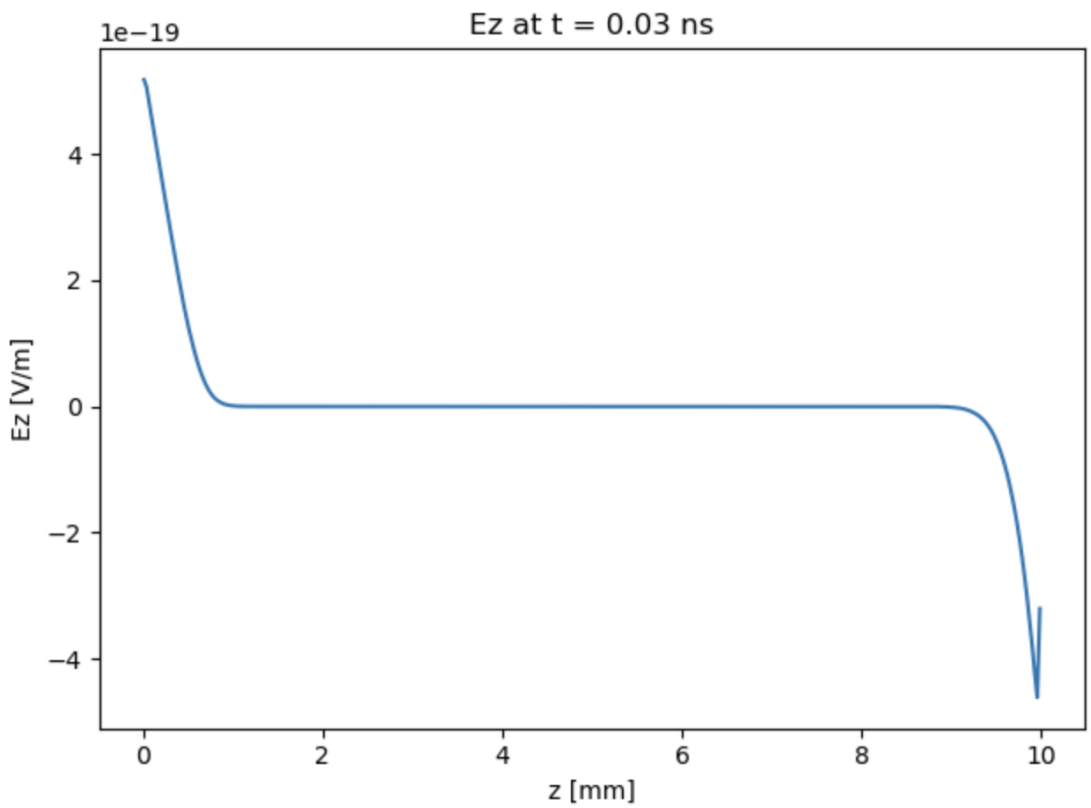

Figure 3: Line-out of E_z vs z at $t \approx 0.03$ ns. Field is positive near the emitting wall and reverses sign near the far boundary, consistent with space-charge build-up. A companion $\phi(z)$ curve can be reconstructed by integrating E_z once Full (HDF5) diagnostics are enabled.

7.3 Phase Space (z–p_z)

```
# save as plot_phase_space.py
import numpy as np, matplotlib.pyplot as plt
from openpmd_viewer import OpenPMDTimeSeries
TS = OpenPMDTimeSeries('diags/diag000100')
it = 2890
# electrons species
z, pz, w = TS.get_particle(['z','uz','w'], species='electrons', iteration=it)
plt.figure(); plt.scatter(z, pz, s=1, alpha=0.3)
plt.xlabel('z [m]'); plt.ylabel('u_z [gamma*beta]'); plt.title(f'Electrons: z-uz at it={
    it}')
plt.grid(True); plt.show()
```

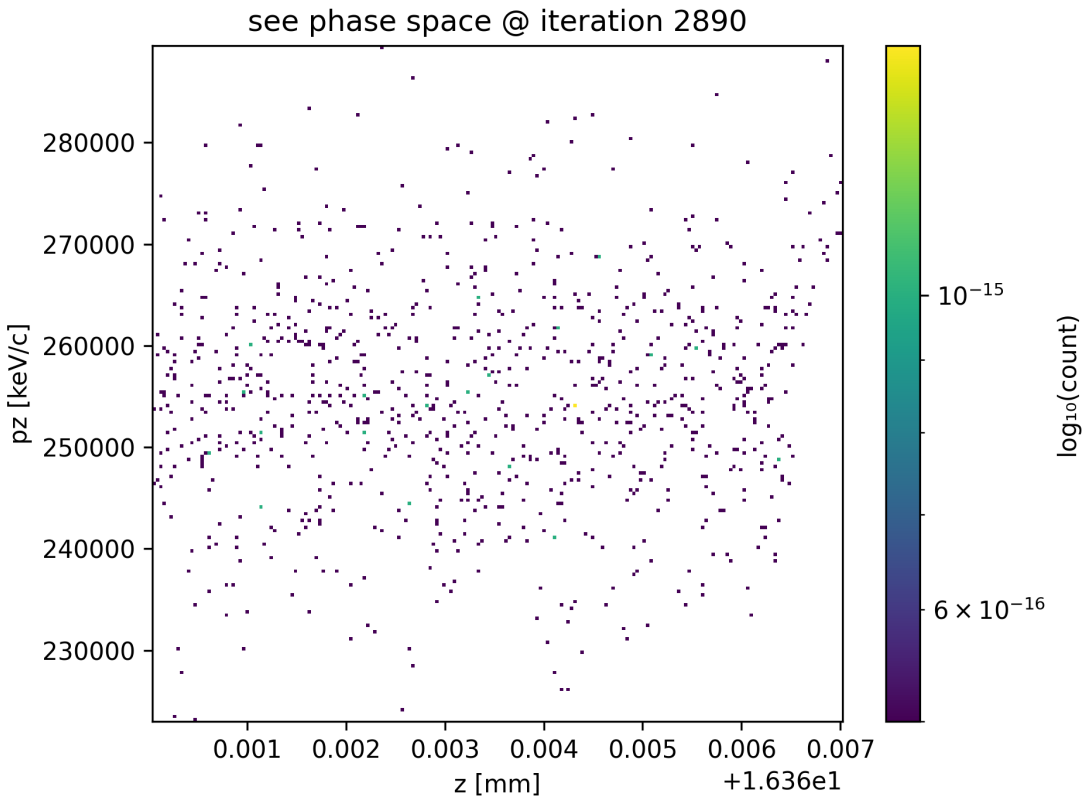

Figure 4: z–p_z distribution for emitted electrons shows a populated near-wall band with initial positive u_z (away from the wall). The spread broadens as space charge develops, a hallmark of approaching the SCL regime.

7.4 Scrape Counts vs. Time

```
# save as plot_scrape_counts.py
import json, glob, numpy as np, matplotlib.pyplot as plt
files = sorted(glob.glob('diags/diag_scrape/openpmd_*.json'))
ts, counts = [], []
for f in files:
    with open(f) as fh:
        j = json.load(fh)
    its = j['iterations']
    for it in its:
        ts.append(it['iteration'])
        counts.append(it['particles_scraped']) # adapt to your schema

plt.figure(); plt.plot(ts, counts)
plt.xlabel('Iteration'); plt.ylabel('Scraped particles'); plt.title('Boundary scrape vs
    iteration')
plt.grid(True); plt.show()
```

SEE activation appears once scraped counts increase from baseline; cadence and triggers set the first visible timestamp.

7.5 ParaView (remote)

If files are large, use **ParaView Client–Server**: run `pvserver` on a compute node and connect from your workstation GUI. HDF5/openPMD readers will show **fields**, isosurfaces, and slices. For 1D, prefer Python scripts; for 2D/3D, ParaView becomes invaluable.

---

8 Troubleshooting & Pitfalls (What Broke & Fixes)

Segfaults with high ppc / high u_z: memory pressure + load imbalance. Reduce ppc, lower emitter flux, or increase MPI ranks; shrink `amr.max_grid_size` to distribute boxes.
ParmParse::sgetarr geometry.prob_lo not found: missing keys or wrong working directory; pass `-i` explicitly and verify relative paths.
"NaN detected": often too-aggressive emission, large Δt, or bad BC mix; back off emission and increase resolution slightly.
Only one figure produced: ensure the loop over iterations matches the diag cadence; confirm diag folder name (`diag000100` vs `diag000050`).

Verification checklist (fast):

- `ts.get_field('Ez', it)` returns sane numbers and `meta.z` is monotonic.
- `diag_scrape/openpmd_*.json` present by $<\sim 500$ iterations at high cadence.
- Re-run 100 steps with $2\times$ ranks $\rightarrow$ time/step drops.

1) "`geometry.prob_lo not found in database`"
Cause: input deck missing required geometry keys or parsed from wrong directory.
Fix: define `geometry.prob_lo` and `geometry.prob_hi` explicitly; ensure `-i <file>` is correct and cwd paths are right.

2) `diag_scrape` starts "late" ($\sim$it=2200)
Cause: large `intervals`; or diag bound to wrong boundary; or JSON not flushed until buffer fills.
Fix: lower `diag_scrape.intervals`; confirm `boundary=zlo`; check JSON schema.

3) Segmentation fault when `num_particles_per_cell` $\sim$500 and $u_z = 1.0$ (large)
Cause: memory pressure from high macro density and injection rate; grid decomposition too coarse.
Fix: reduce ppc; coarsen injection flux; increase MPI ranks; decrease `amr.max_grid_size` to distribute memory.

4) Plotting inversion bug
Cause: axis interpretation (x vs z) or sign convention mix-up.
Fix: pull coordinates from metadata (e.g., `meta.z`); verify sign of E_z; define $\phi(0) = 0$ reference.

5) HDF5 missing $\Rightarrow$ blocked diagnostics
Cause: built without `-DopenPMD_USE_HDF5=ON`.
Fix: rebuild WarpX with HDF5; set `warpx.openpmd_backend=h5`.

6) `yt` / Python import errors on Puma
Fix: use site module `hpcteam/pywarpx/25.03` or install in conda env. Verify with a one-line import.

7) MPI fabric issues
Fix: set `OMPI_MCA_pml=ob1` and `OMPI_MCA_btl=self,tcp` (or switch to UCX properly if site supports it). Start with `OMP_NUM_THREADS=1`.

8) Only one figure generated / wrong iteration
Fix: loop over iterations; ensure diag path matches; confirm output cadence (`diagnostics.intervals`).

---

9 Reproducible Run

9.1 Minimal 1D SEE Sheath (fast turnaround)

1. Copy `inputs_sheath_1d` from this doc (adjust z-extent, resolution).
2. Set modest `num_particles_per_cell` (e.g., 50) and emission flux.
3. Run to **max_step** ∼**5k**; write diags every 50–100 iters.
4. Plot E_z and ϕ. Confirm expected sign and near-wall behavior as you vary flux (r).

9.2 Scaling Up

- Increase resolution (n_cell) and ppc gradually.
- Re-tune `amr.max_grid_size` for better load balance.
- Use more MPI ranks rather than large OMP threads at first.

9.3 Long Run (10k iterations)

- Verify stability at 2k/5k first; then extend to 10k to probe (quasi-)equilibria and virtual cathode formation.
- Consider writing reduced diags more frequently early, then thin out cadence later to save I/O.

---

10 Folder & File Keeping

```
project/
  inputs_sheath_1d
  slurm/SEE1D.slurm
  diags/
    diag000100/ # Full diags (HDF5)
    diag_scrape/ # Reduced JSON
  plots/
    Ez_phi_it3000.png
    phase_it2890.png
  scripts/
    plot_fields_phi.py
    plot_phase_space.py
    plot_scrape_counts.py
  restart/
```

Conventions that help: timestamp your plots (Ez_0003000.png), add a README with run hash (git commit of inputs), and note the WarpX version and backend (HDF5/ADIOS2). Keep post-processing scripts alongside the run in `scripts/` and make them path-agnostic (`$PWD`).

---

11 Documentation Figures

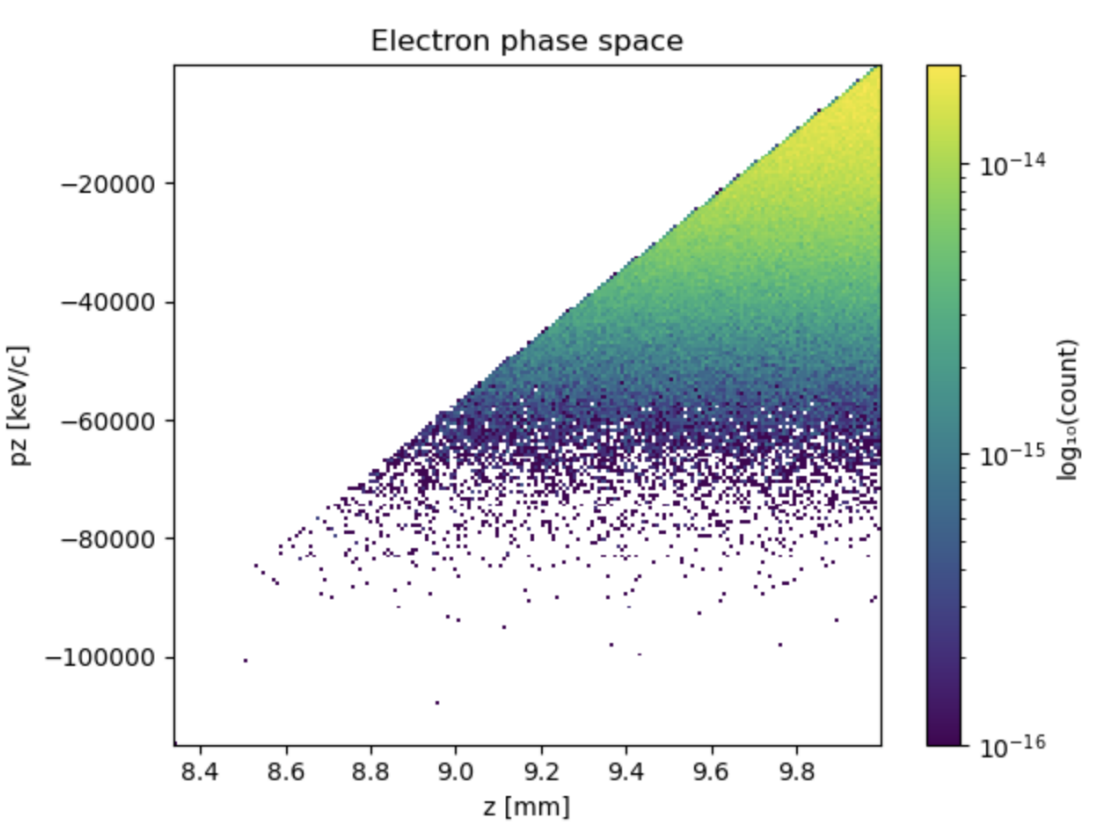

Figure 5: A triangular upper envelope in z–p_z indicates an accelerating near-wall cohort; the dense band thickens as emission ramps.

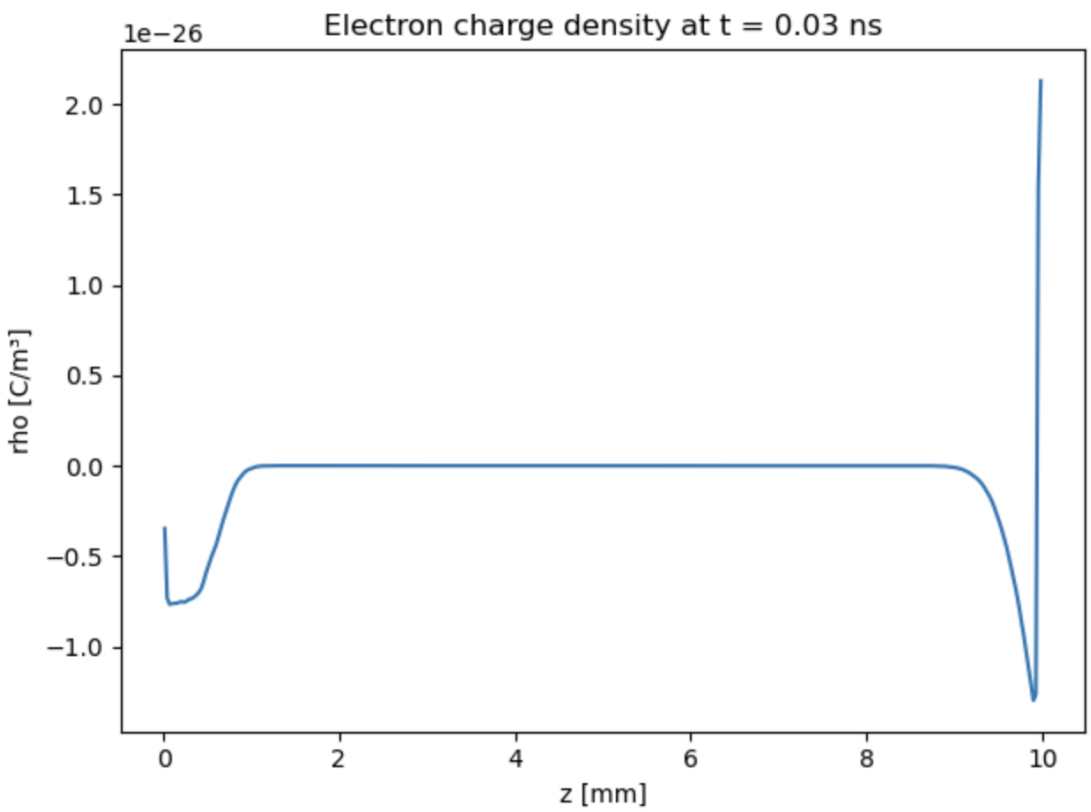

Figure 6: Near-wall negative spike and slight plateau in the bulk; magnitudes are consistent with small-signal sheath formation at early times.

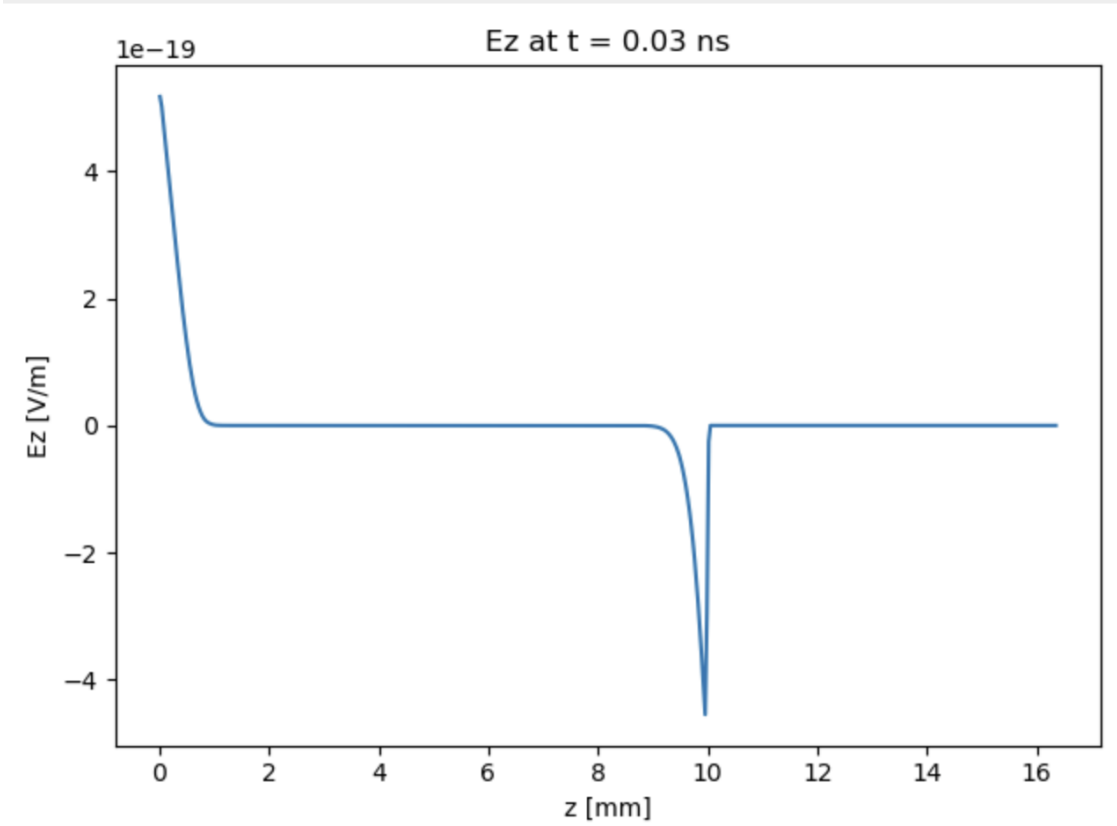

Figure 7: Extending the domain displaces the reversal region further from the wall, clarifying the far-boundary influence.

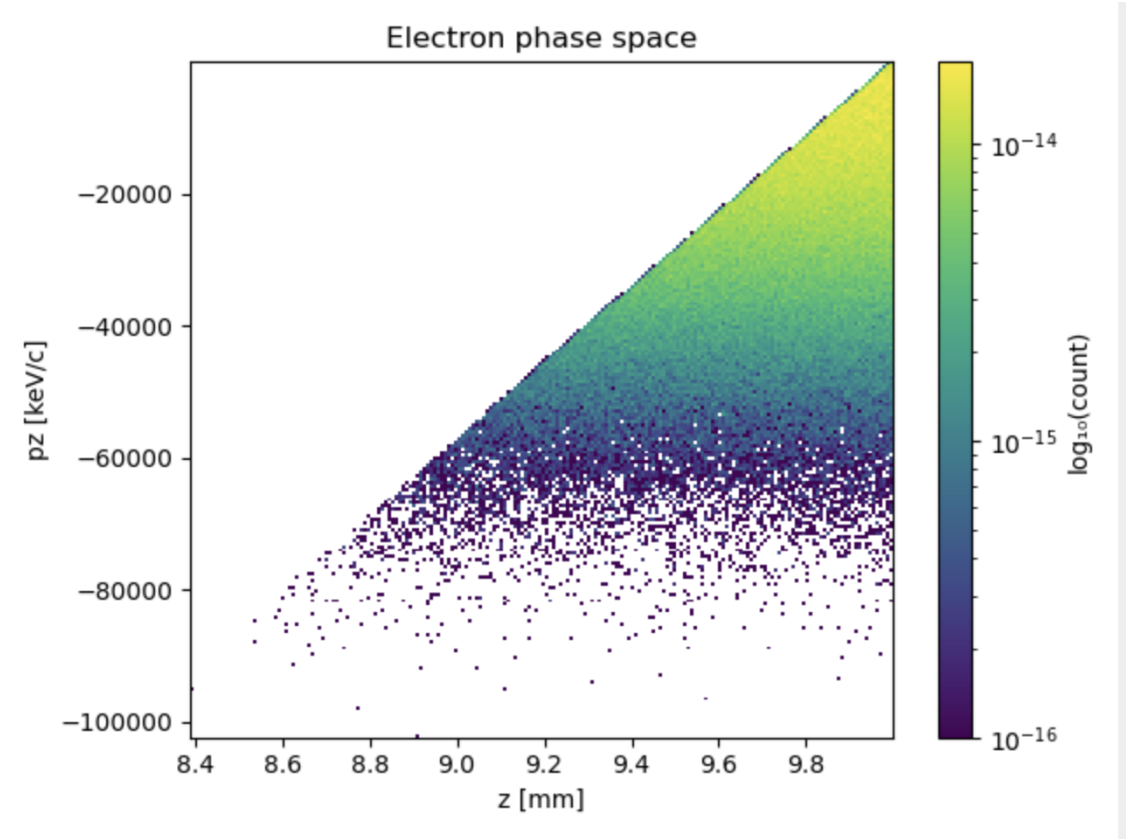

Figure 8: Broader momentum spread reflects cumulative space-charge effects and partial trapping.

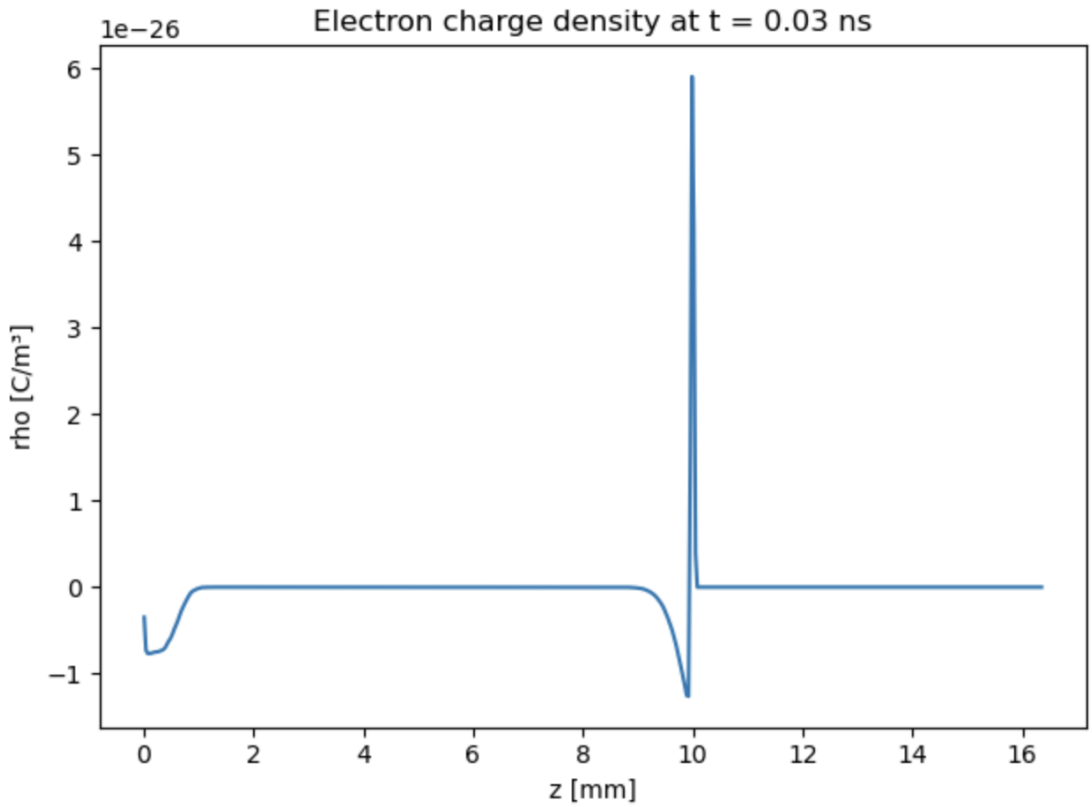

Figure 9: Pronounced spike near $z \approx 10\,\mathrm{mm}$ aligns with the strong negative E_z lobe in Fig. 10; together they indicate a developing virtual-cathode region.

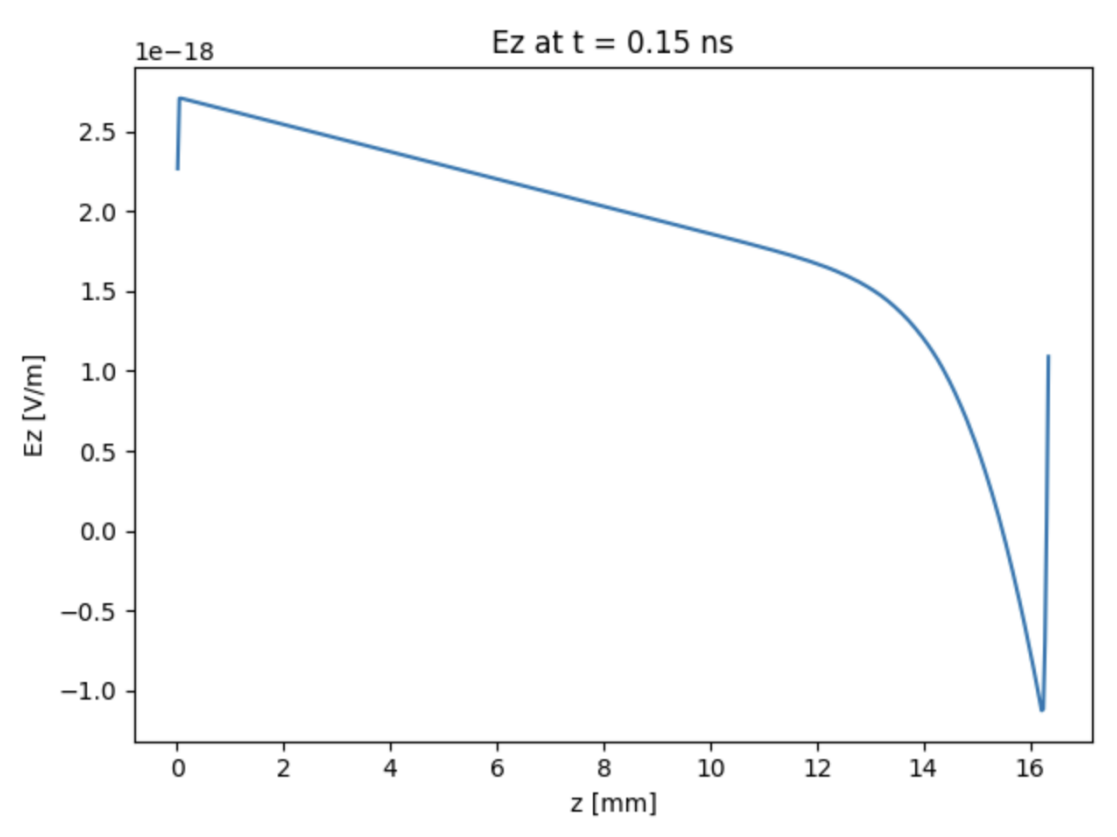

Figure 10: Gradual decline with a strong negative lobe near the far boundary, indicating accumulated space charge and partial field reversal.

---

12 Open Questions & Next Steps

- Rebuild with HDF5 (if not already) to unlock complete potential/charge diags for rigorous $\phi(z)$ reconstruction.
- 10k-iteration surveys across 2–3 emission levels to map transition into SCL; record scrape and E_z every 25 iterations early.
- Parameter sweeps: (ppc, u_z, flux) one-at-a-time, plus a coarse grid (2×3) to find the knee where field reversal first appears.
- Questions for Remi: best-practice keys for surface emission blocks by version; recommended reduced diags for current neutrality accounting; example decks for biased walls.
- **Rebuild with HDF5** (if not already) to unlock complete Full diags and smoother yt/ParaView workflows.
- **10k-iteration** runs to check for steady or oscillatory equilibria; track r trend.
- **Time-step refinement**: adjust output cadence for clearer temporal transitions (e.g., $\Delta it = 25$ early, $\Delta it = 200$ late).
- If pivoting, collect a **question list for Remi** (diagnostic schema, best practices for SEE sources, recommended yield models per energy/angle, etc.).

---

13 Appendix A — Example "Everything in One Place" Slurm

```
#!/bin/bash
#SBATCH --job-name=SEE_full
#SBATCH --account=hanquist
#SBATCH --partition=standard
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=8
#SBATCH --mem-per-cpu=4G
#SBATCH --time=02:00:00
#SBATCH -o %x.out
#SBATCH -e %x.err

module load warpx/25.03
export OMPI_MCA_pml=ob1
export OMPI_MCA_btl=self,tcp
export OMP_NUM_THREADS=1

cd $SLURM_SUBMIT_DIR

BIN=$(which warpx)
INPUT=inputs_sheath_1d

# Short smoke test
echo "[SMOKE]"; srun -n 1 $BIN -i $INPUT max_step=50 || exit 1

# Main run
echo "[RUN]"; srun -n $SLURM_NTASKS $BIN -i $INPUT max_step=5000 || exit 2

# Postproc (first-pass)  safe to run on login if light
echo "[POST]"; python3 scripts/plot_fields_phi.py || true
python3 scripts/plot_phase_space.py || true
python3 scripts/plot_scrape_counts.py || true
```

---

14 Appendix B — Common Errors (with fixes)

- **NaNs in fields/particles**: Lower time step (implicit via CFL), reduce ppc, verify boundary mixes, ensure emitter model parameters are physically plausible.
- `amrex.abort_on_out_of_gpu_memory`: Reduce ppc, shrink domain or increase MPI ranks, check GPU allocation (if on GPU nodes).
- `module: command not found`: On systems without environment modules; load site profile or use Conda.
- `yt` missing: Install in env or use site Python module.

Acknowledgments

Thanks to Dr. Hanquist and CHANL members for guidance; thanks to Remi (WarpX) and HPC support (Puma) for tooling help. This doc aggregates lessons learned so the next person can move faster with fewer dead ends.